

Universidad Nacional de Ingeniería
Facultad de Ingeniería Industrial y de Sistemas
Escuela Profesional de Ingeniería de Software
SW505-Construcción de Software I
Docente Mag. Juan Luis Herencia Guerra

1ra Práctica Calificada

Código de Alumno: _____ Apellidos y nombres: _____

Duración: 90 minutos (1.5 horas)

Instrucciones: La práctica se realizará en computadora, en grupos de dos. Cree un folder en su repositorio de Github llamado "PC1-Apellido 1er participante-Apellido 2do participante" y suba los problemas resueltos 10 minutos antes de finalizar la práctica.

Parte I

Objetivos:

- **Diagnosticar** problemas de cohesión y acoplamiento en un sistema monolítico real.
- **Separar responsabilidades** identificando qué cambia junto y qué debe vivir aparte.
- **Diseñar interfaces internas** que aislen el cambio y permitan crecer el sistema sin reescribirlo.
- **Extender** el sistema con un nuevo módulo (matrícula por período) sin tocar el código existente más de lo indispensable.

Historia del Caso

El siguiente sistema fue construido por un equipo que ya no está. Originalmente registraba alumnos, docentes y cursos, y funcionaba... hasta que el negocio pidió algo aparentemente simple:

"Necesitamos registrar matrículas cada período: 2026-1, 2026-2, Verano 2027, y conservar el historial completo."

El equipo descubrió que el sistema no podía crecer sin reescribirlo. Un solo archivo, una sola clase, todo mezclado. Este es tu punto de partida.

Código del sistema

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class SistemaAcademico {
    // "Base de datos" en memoria: todo revuelto en listas paralelas
    static List<String> alumnos = new ArrayList<>();
    static List<String> docentes = new ArrayList<>();
    static List<String> cursos = new ArrayList<>();
    static List<String> matriculas = new ArrayList<>(); // formato: "codA
Lumno|codCurso|periodo"

    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n=== SISTEMA ACADÉMICO ===");
            System.out.println("1. Registrar alumno");
            System.out.println("2. Registrar docente");
            System.out.println("3. Registrar curso");
            System.out.println("4. Matricular alumno en curso");
            System.out.println("5. Listar todo");
            System.out.println("6. Salir");
            System.out.print("Opción: ");

            int op = Integer.parseInt(sc.nextLine());

            if (op == 1) {
                System.out.print("Código alumno: ");
                String cod = sc.nextLine();
                System.out.print("Nombre: ");
                String nom = sc.nextLine();
                System.out.print("Carrera: ");
                String car = sc.nextLine();
```

```
// Validación débil: si no valida formato, se guarda igual
            if (cod.isEmpty() || nom.isEmpty()) {
                System.out.println("Error: datos incompletos");
            } else {
                alumnos.add(cod + "|" + nom + "|" + car);
                System.out.println("Alumno registrado.");
            }
        }
        else if (op == 2) {
            System.out.print("Código docente: ");
            String cod = sc.nextLine();
            System.out.print("Nombre: ");
            String nom = sc.nextLine();
            System.out.print("Especialidad: ");
            String esp = sc.nextLine();
            if (cod.isEmpty() || nom.isEmpty()) {
                System.out.println("Error: datos incompletos");
            } else {
                docentes.add(cod + "|" + nom + "|" + esp);
                System.out.println("Docente registrado.");
            }
        }
        else if (op == 3) {
            System.out.print("Código curso: ");
            String cod = sc.nextLine();
            System.out.print("Nombre: ");
            String nom = sc.nextLine();
            System.out.print("Créditos: ");
            int cred = Integer.parseInt(sc.nextLine());
            System.out.print("Código docente: ");
            String codDoc = sc.nextLine();
            // Sin validar que el docente exista
            cursos.add(cod + "|" + nom + "|" + cred + "|" + codDoc);
            System.out.println("Curso registrado.");
        }
        else if (op == 4) {
            System.out.print("Código alumno: ");
            String codAl = sc.nextLine();
            System.out.print("Código curso: ");
            String codCur = sc.nextLine();
            // PROBLEMA: no hay período. Si el alumno lleva el mismo

            // en 2026-1 y 2026-2, se duplica sin distinción.
            matriculas.add(codAl + "|" + codCur + "|SIN_PERIODO");
            System.out.println("Matriculado.");
        }
        else if (op == 5) {
            System.out.println("\n--- ALUMNOS ---");
            for (String a : alumnos) System.out.println(a);
            System.out.println("--- DOCENTES ---");
            for (String d : docentes) System.out.println(d);
            System.out.println("--- CURSOS ---");
            for (String c : cursos) System.out.println(c);
            System.out.println("--- MATRÍCULAS ---");
            for (String m : matriculas) System.out.println(m);
        }
        else if (op == 6) {
            System.out.println("Adiós.");
            break;
        }
        else {
            System.out.println("Opción inválida.");
        }
    }
}
```

Responder en el cuadernillo

1.1 Cohesión y acoplamiento

Responde brevemente:

1. **¿Cuántas responsabilidades distintas tiene la clase SistemaAcademico?** Enumera al menos 6.
2. **¿Qué tipo de cohesión tiene esta clase?** (funcional, secuencial, comunicacional, lógica, coincidente). Justifica.

3. **¿Qué pasa si cambia el formato de almacenamiento** (por ejemplo, de "cod|nom" a JSON)? ¿Cuántos lugares del archivo hay que tocar?
4. **¿Qué pasa si mañana se agrega "registrar horarios"?** ¿Cuántas líneas nuevas tendría que meter en main?

1.2 Separación de responsabilidades

Identifica al menos **tres responsabilidades** que están mezcladas en el código y que deberían vivir en módulos separados. Ejemplo de formato:

Responsabilidad	Dónde está ahora	Dónde debería estar
Entrada/salida por consola	main, mezclada con lógica	Clase VistaConsola
...	...	...

1.3 Abstracción e interfaces internas

1. **¿Qué "contratos" implícitos existen en el código?** (por ejemplo: "el formato de una matrícula es codAlumno|codCurso").
2. **¿Por qué esos contratos están rotos?** Piensa en la matrícula que ahora necesita un período.
3. **Propón al menos dos interfaces internas** que permitirían aislar el cambio.

Parte 2 — Refactorización y nuevo módulo

Meta: Entregar un sistema modular con matrícula por período, conservando el historial.

Requisitos funcionales

1. **Registrar matrícula** especificando alumno, curso y período (ej. "2026-2", "Verano 2027").
2. **Listar matrículas por período.**
3. **Listar todas las matrículas de un alumno** (historial completo).
4. **Listar el historial de un curso** (todos los períodos en que se dictó).
5. **Impedir doble matrícula** del mismo alumno en el mismo curso y período.
6. Los módulos de **alumnos, docentes y cursos** siguen funcionando igual.

Requisitos de diseño (obligatorios)

- **Al menos 5 clases** con responsabilidad única y nombre claro.
- **Al menos 2 interfaces** que representen abstracciones internas (ej. RepositorioAlumno, RepositorioMatricula).
- **Cero cadenas "cod|nom|..." usadas como almacenamiento:** cada entidad es un objeto (Alumno, Docente, Curso, Matricula).
- **Cero lógica de negocio dentro del main:** el main solo orquesta la vista.
- **Un módulo de matrícula separado** que encapsule el período y la validación de duplicados.
- **Sin usar frameworks ni librerías externas.** Solo Java estándar.

Estructura sugerida (puedes cambiarla, pero justificala)

```
src/
├── Main.java
├── modelo/
│   └── Alumno.java
```

```
├── Docente.java
├── Curso.java
├── Matricula.java
├── repositorio/
│   ├── RepositorioAlumno.java      (interfaz)
│   ├── RepositorioDocente.java     (interfaz)
│   ├── RepositorioCurso.java       (interfaz)
│   ├── RepositorioMatricula.java   (interfaz)
│   └── impl/
│       ├── RepositorioAlumnoMemoria.java
│       ├── RepositorioDocenteMemoria.java
│       ├── RepositorioCursoMemoria.java
│       └── RepositorioMatriculaMemoria.java
├── servicio/
│   ├── ServicioAlumno.java
│   ├── ServicioDocente.java
│   ├── ServicioCurso.java
│   └── ServicioMatricula.java      ← aquí vive la nueva
├── lógica
├── vista/
│   └── VistaConsola.java
```

Pistas para el nuevo módulo de matrícula

Modelo Matricula:

```
public class Matricula {
    private final String codigoAlumno;
    private final String codigoCurso;
    private final String periodo;    // "2026-2",
    "Verano 2027"
    private final LocalDate fecha;

    // constructor, getters, equals/hashCode por
    (alumno, curso, periodo)
}
```

Interfaz RepositorioMatricula:

```
public interface RepositorioMatricula {
    void guardar(Matricula m);
    boolean existe(String codigoAlumno, String
codigoCurso, String periodo);
    List<Matricula> porPeriodo(String periodo);
    List<Matricula> porAlumno(String codigoAlumno);
    List<Matricula> porCurso(String codigoCurso);
}
```

Servicio:

```
public class ServicioMatricula {
    private final RepositorioMatricula repositorio;
    private final RepositorioAlumno repositorioAlumno;
    private final RepositorioCurso repositorioCurso;

    public ServicioMatricula(RepositorioMatricula r,
RepositorioAlumno ra,
RepositorioCurso rc) { /*
DI por constructor */ }

    public void matricular(String codAlumno, String
codCurso, String periodo) {
        // 1. Validar que alumno y curso existan
        // 2. Validar que no exista matrícula previa en
el mismo período
        // 3. Guardar la matrícula
    }
}
```

Vista (opción nueva del menú):

7. Matricular en período
8. Listar matrículas por período
9. Ver historial de un alumno
10. Ver historial de un curso

Ejemplo de ejecución esperada

=== SISTEMA ACADÉMICO ===

7. Matricular en período

...

Opción: 7

Código alumno: A001

Código curso: C010

Período (ej. 2026-2): 2026-2

Matrícula registrada: A001 en C010 (2026-2)

Opción: 7

Código alumno: A001

Código curso: C010

Período (ej. 2026-2): Verano 2027

Matrícula registrada: A001 en C010 (Verano 2027)

Opción: 7

Código alumno: A001

Código curso: C010

Período (ej. 2026-2): 2026-2

ERROR: El alumno A001 ya está matriculado en C010 para el período 2026-2